COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.4 · Application Attacks

Mestre CyberSec · **Lesson 2.4 — Application Attacks** (11:48) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Aplicação = a superfície mais atacada: privilege escalation (horizontal = mesmo nível, outro dado; vertical = tier acima — user vira admin); directory traversal (../../etc/passwd); remote code execution. Defesa: input validation, patching, WAF, SCA (library vulnerable), secure co

💡 Key Exam Highlights & Critical Concepts

- Privesc horizontal (mesmo nível, outro dado) vs vertical (tier acima)
- Directory traversal: sobe com ../ pra /etc/passwd
- Defesa: input validation SEMPRE; nunca confie no input

🎯 Exam Tips & Typical CompTIA Scenarios

- 'user gains higher-level access' = privilege escalation (vertical)
- 'path traversal reads arbitrary files' = directory traversal

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.4 — Application Attacks)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• Privilege Escalation (PE)

An attack where a user gains unauthorized access levels within a system. Vertical escalation moves to a higher privilege tier (e.g., user to admin); horizontal escalation accesses another account at the same privilege level.

⚠ ***Exam Trap:*** *If the question says 'user gains higher-level access' or 'user becomes admin,' it is vertical privilege escalation. Horizontal = same level, different data/account.*

• Directory Traversal (DT)

An injection attack using sequences like '../' (dot-dot-slash) in a file path input to escape the intended directory and read arbitrary files on the server, such as /etc/passwd or config files.

⚠ ***Exam Trap:*** *Look for clues like '../', 'path traversal', or 'arbitrary file read.' The classic target file on Linux is /etc/passwd. Mitigation = input validation blocking '../' sequences.*

• Remote Code Execution (RCE)

A critical vulnerability where an attacker remotely runs arbitrary commands or code on the target system, typically through injection flaws, deserialization bugs, or unpatched services.

⚠ ***Exam Trap:*** *RCE is the highest-severity outcome. Watch for terms 'arbitrary code execution,' 'shell access,' or 'command injection' leading to full system compromise.*

• Input Validation (IV)

A secure coding practice that verifies, sanitizes, and constrains all user-supplied input against an allowlist of expected values before processing, preventing injection and traversal attacks.

⚠ ***Exam Trap:*** *The golden rule: 'Never trust user input.' Input validation is the first-line defense against SQLi, XSS, directory traversal, and command injection — expect it as the answer whenever these attacks appear.*

• Web Application Firewall (WAF)

A specialized Layer 7 firewall that inspects HTTP/HTTPS traffic to detect and block web-based attacks such as SQL injection, XSS, and directory traversal against web applications.

⚠️ ***Exam Trap:*** *Distinguish WAF (Layer 7, protects web apps from HTTP-layer attacks) from NGFW (broader Layer 3-7 network protection). WAF = OWASP Top 10 shield.*

• Software Composition Analysis (SCA)

A security tool that scans an application's third-party libraries, dependencies, and open-source components against vulnerability databases to identify known CVEs and licensing issues.

⚠️ ***Exam Trap:*** *SCA = third-party/library scanning (Log4Shell-style vulnerabilities). Do not confuse with SAST (scans YOUR source code) or DAST (runtime black-box testing).*

• SQL Injection (SQLi)

An injection attack where malicious SQL statements are inserted into an input field to manipulate the backend database, potentially exposing, altering, or deleting data.

⚠️ ***Exam Trap:*** *Always pair SQLi mitigation with 'parameterized queries / prepared statements.' Input validation alone is NOT enough — keywords like 'UNION SELECT' or 'OR 1=1' are red flags.*

• Cross-Site Scripting (XSS)

An injection flaw where malicious scripts are injected into a trusted web page and executed in the victim's browser. Stored XSS persists on the server; reflected XSS is delivered via a crafted link.

⚠️ ***Exam Trap:*** *XSS targets the user's browser/client (not the server). Mitigation = output encoding and Content Security Policy (CSP). If the question mentions 'stealing cookies' or 'session hijacking via script,' think XSS.*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 Server-Side Request Forgery (SSRF)

🏠 **Real-World Analogy:** SSRF is like standing outside a high-security embassy and handing a task memo to the official internal courier: 'Please step into the classified vault on floor 8, photograph the ambassador's ledger, and bring the printout to me at the curbside desk'. Because the courier holds clearance to navigate interior corridors inaccessible to the general public, they faithfully retrieve the classified internal file and hand it directly to the outsider.

🔍 **Root Cause & Background:** Web applications frequently fetch remote resources (importing images or fetching webhooks). If the server accepts unvalidated user-supplied destination URLs, attackers supply internal IP targets (loopback `127.0.0.1`, RFC 1918 private subnets) or cloud instance metadata endpoints (`169.254.169.254`), exfiltrating ephemeral IAM credentials.

🛡️ **GRC Auditor Perspective:** Enforce strict destination allowlists on server outbound calls; disable alternative URI schemes (e.g., `file://`); block loopback and private IP resolution; and mandate cloud Instance Metadata Service Version 2 (IMDSv2) requiring session tokens.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker manipulates a web app into issuing unauthorized requests targeting internal private addresses or cloud metadata (169.254.169.254)' -> Server-Side Request Forgery (SSRF).

📌 Cross-Site Request Forgery (CSRF)

🏠 **Real-World Analogy:** You have an active, authenticated tab open with your online bank. In another browser tab, you visit an untrusted forum or deceptive link. That third-party site stealthily fires an HTTP POST request to your bank: 'Transfer $1,000 to adversary account 999'. Because your web browser automatically includes your valid session cookies with any request addressed to that bank, the banking server assumes you intentionally triggered the action and executes the wire.

🔍 **Root Cause & Background:** Browsers automatically attach ambient session cookies whenever dispatching requests to target origins. Without anti-forgery validation tokens tied to specific form renders, servers cannot discern whether an action was deliberately initiated by the user or forged across origins.

🛡️ **GRC Auditor Perspective:** Mandate cryptographically random Anti-CSRF Synchronizer Tokens embedded within state-changing forms; enforce the `SameSite=Strict` or `SameSite=Lax` attribute on session cookies; and require step-up reauthentication for critical actions.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker leverages an authenticated victim session to force unauthorized actions on a trusted website' -> Cross-Site Request Forgery (CSRF). Primary defenses: Anti-CSRF Tokens and SameSite cookies.

📌 Broken Object Level Authorization (BOLA) & API Security

🏠 **Real-World Analogy:** BOLA (historically IDOR) is like renting locker 101 at a gym. After your workout, you walk past locker 102, jiggle the handle, and the door swings open, exposing another member's valuables. The gym verified you are a paying member permitted inside the facility (valid authentication), but completely neglected to verify whether you hold the legal authority to open locker 102 specifically.

🔍 **Root Cause & Background:** In modern REST APIs, clients reference data objects via request parameters (e.g., `/api/records/402`). While the API validates that the client holds an active auth token (Authentication), it fails to check if that user is legitimately authorized to access object 402 (Authorization). BOLA ranks as the #1 threat in the OWASP API Security Top 10.

🛡️ **GRC Auditor Perspective:** Audit API routes to ensure object-level ownership checks (ABAC) run prior to database record retrieval; substitute sequential integer IDs with cryptographically random UUIDv4s; and deploy API Gateways enforcing centralized policy checks.

⚠️ **How CompTIA Tests This:** Trigger: 'authenticated user changes an ID parameter in an API call from 2001 to 2002 and views another customer's confidential statement' -> Broken Object Level Authorization (BOLA) / IDOR.

📌 HTTP Parameter Pollution (HPP)

🏠 **Real-World Analogy:** It is like completing an invoice voucher that features a 'Payee:' line, and writing the payee twice: on the top line you write the vetted vendor name, but on the bottom line you write the fraudster's name. The front-desk inspector (the WAF) reviews only the first line, approving the voucher; the disbursement cashier (the backend database) reads only the last line, sending the funds to the fraudster. Discrepancies between how components parse identical keys create security bypasses.

🔍 **Root Cause & Background:** Web application frameworks handle duplicate HTTP parameters inconsistently (some parse only the first value, others only the last, while others concatenate them). Adversaries exploit these parsing mismatches between perimeter WAFs and backend servers (`?role=user&role=admin`) to bypass validation filters.

🛡️ **GRC Auditor Perspective:** Standardize parameter parsing logic across front-end reverse proxies and backend servers; reject requests containing duplicate parameter occurrences; and enforce strict OpenAPI input schemas.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker injects duplicate parameters with identical names to bypass security filters caused by parsing differences between the WAF and backend' -> HTTP Parameter Pollution (HPP).

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Attacker coerces a web application into making unauthorized requests to internal IP addresses or the cloud metadata service	**Server-Side Request Forgery (SSRF)**
Malicious site tricks an authenticated browser into submitting state-changing transactions against a trusted application	**Cross-Site Request Forgery (CSRF)**
Authenticated client alters an ID parameter in an API endpoint request to access unauthorized records belonging to others	**Broken Object Level Authorization (BOLA) / IDOR**
Attacker supplies duplicate parameter names in an HTTP query to bypass WAF validation filters due to parsing differences	**HTTP Parameter Pollution (HPP)**
Defensive countermeasure attaching unique unpredictable tokens to forms ensuring only deliberate user actions execute	**Anti-CSRF Token (Synchronizer Token)**
AWS metadata service version requiring session token generation via HTTP PUT requests, neutralizing SSRF attempts	**IMDSv2 (Instance Metadata Service Version 2)**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A security analyst reviews web server logs and sees repeated requests for URLs such as /download?file=../../../../etc/passwd and /download?file=..%2f..%2fconfig.php. The application is returning contents of files outside the intended directory. Which application attack is occurring?

A) SQL injection

B) Cross-site scripting

C) Directory traversal

D) Privilege escalation

QUESTION 2 · EXAM SCENARIO

A web application reflects the value of a search parameter directly into the HTML response without encoding. An attacker submits a search term containing a crafted script tag, and the script executes in other users' browsers when they view the search results. Which vulnerability is being exploited?

A) Reflected cross-site scripting

B) SQL injection

C) Directory traversal

D) Remote code execution

QUESTION 3 · EXAM SCENARIO

A login form builds a database query by concatenating the username and password fields directly into the SQL statement. An attacker enters a specially crafted username that changes the query logic and bypasses authentication. Which attack is this?

A) Cross-site request forgery

B) SQL injection

C) Directory traversal

D) Cross-site scripting

QUESTION 4 · EXAM SCENARIO

A user logged into an online banking application changes the account ID parameter in a request from their own account number to another customer's account number. The application returns the other customer's account details even though both users have the same role. Which type of privilege escalation is this?

A) Vertical privilege escalation

B) Horizontal privilege escalation

C) Directory traversal

D) Remote code execution

QUESTION 5 · EXAM SCENARIO

An attacker with a standard user account manipulates a hidden parameter in a profile update request. The application then grants the account access to administrative functions that are normally restricted to IT staff. Which type of privilege escalation has occurred?

A) Horizontal privilege escalation

B) Vertical privilege escalation

C) SQL injection

D) Cross-site scripting

QUESTION 6 · EXAM SCENARIO

A web application allows users to upload image files. After a file is uploaded, the server processes its metadata using a library that invokes an external command. An attacker embeds shell commands in the metadata, and the server executes them with the web service account's permissions. Which vulnerability has been exploited?

A) Cross-site scripting

B) Directory traversal

C) Remote code execution

D) Software composition analysis

QUESTION 7 · EXAM SCENARIO

A development team wants to identify known vulnerabilities in the open-source libraries and frameworks used by its applications. The team needs a tool that inventories dependencies and maps them to known CVEs. Which type of analysis should be used?

A) Dynamic application security testing

B) Software composition analysis

C) Web application firewall

D) Static application security testing

QUESTION 8 · EXAM SCENARIO

An organization wants to protect a public web application from common injection attacks before the requests reach the application servers. The security team needs a control that inspects HTTP traffic and can block malicious patterns. Which control best meets this requirement?

A) Software composition analysis

B) Web application firewall

C) Input validation

D) Patch management

QUESTION 9 · EXAM SCENARIO

During a code review, a security engineer finds that a web form relies only on JavaScript checks in the browser to validate age and postal code fields. Attackers can bypass these checks and submit malicious values directly to the server. Which defense should be added?

A) Web application firewall

B) Server-side input validation

C) Software composition analysis

D) Directory traversal prevention

QUESTION 10 · EXAM SCENARIO

A file download feature constructs a file path from a user-supplied parameter. Developers want to prevent attackers from accessing files outside the intended download directory. Which control should be implemented?

A) Encode all HTML output before rendering pages

B) Canonicalize the path and validate it against an allowlist or base directory

C) Use parameterized database queries for all file metadata lookups

D) Run dynamic application security testing against the staging environment

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option C — Directory traversal

Directory traversal uses ../ sequences or encoded variants to break out of the intended directory and access files outside the web root. SQLi targets database queries, XSS targets browser execution, and privilege escalation gains higher access rights; the file path manipulation is the key indicator.

Question 2: Option A — Reflected cross-site scripting

Reflected XSS occurs when user input is immediately returned in the response without output encoding, allowing scripts to run in the victim's browser. SQLi affects database queries, directory traversal accesses files, and RCE executes code on the server.

Question 3: Option B — SQL injection

SQL injection occurs when untrusted input alters the structure of a SQL query. In this case, the crafted username changes the query condition to bypass authentication. CSRF abuses authenticated sessions, directory traversal manipulates file paths, and XSS executes scripts in browsers.

Question 4: Option B — Horizontal privilege escalation

Horizontal privilege escalation occurs when a user accesses resources belonging to another user at the same privilege level. Vertical escalation would involve gaining higher privileges, such as becoming an administrator.

Question 5: Option B — Vertical privilege escalation

Vertical privilege escalation occurs when a user gains a higher level of access, such as moving from standard user to administrator. Horizontal escalation would involve accessing another user's resources at the same level.

Question 6: Option C — Remote code execution

Remote code execution allows an attacker to run arbitrary commands or code on the server. The embedded shell commands executed by the server indicate RCE. XSS runs in the browser, directory traversal reads files, and SCA is a dependency analysis process, not an attack.

Question 7: Option B — Software composition analysis

Software composition analysis (SCA) examines third-party components and dependencies to identify known vulnerabilities and license issues. DAST tests running applications, SAST analyzes source code, and a WAF blocks traffic at runtime.

Question 8: Option B — Web application firewall

A web application firewall (WAF) inspects HTTP/HTTPS traffic and can block common attacks such as SQL injection and XSS before they reach the application. SCA analyzes dependencies, input validation is implemented in the application, and patch management fixes known software flaws.

Question 9: Option B — Server-side input validation

Server-side input validation ensures that data is validated on the server regardless of client-side checks, which can be bypassed. A WAF can help but does not replace secure input handling; SCA and directory traversal prevention address other issues.

Question 10: Option B — Canonicalize the path and validate it against an allowlist or base directory

Canonicalizing the path and validating it against an allowlist or base directory prevents directory traversal by resolving encoded and relative path sequences before file access. Output encoding addresses XSS, parameterized queries address SQL injection, and DAST is a testing method, not a preventive control.